

HyperBus - Technical Specification Document

1. Front Matter

Title: HyperBus Bus Ticketing Application Technical Specification

Author: Vishwa

Version: 1.0.0

2. Introduction

a. Overview

HyperBus is a bus ticketing platform that allows users to seamlessly search, book, and manage bus trips. The system includes functionality for user roles (passengers, operators, admins), secure payments, and real-time seat selection.

b. Goals

Product Requirements:

User registration and authentication with role-based access

Bus search functionality with filters (source, destination, date)

Realtime seat selection and booking

Secure payment processing

Booking management (view, cancel)

Operator management of buses and trips

Administrative oversight and user management

Rating and feedback system

Payment Integration

Technical Requirements:

RESTful API architecture

JWT based authentication

MongoDB database with Mongoose ODM

Express.js backend framework

Docker containerization

Comprehensive logging and monitoring

Unit and load testing coverage using Jest and K6

CI/CD integration for automated build, test and deploy workflows

Centralized logging and observability using ELK Stack (Elasticsearch, Logstash, Kibana)

3. Solutions

a. Proposed Solution

Architecture

- **Backend:** Node.js + Express.js
- **Database:** MongoDB with Mongoose
- **Auth:** JWT (with refresh tokens) + bcrypt
- **Payment:** Razorpay

- **Infrastructure:** Docker + Nginx reverse proxy + HTTPS
- **Monitoring & Logging:**
 1. Log Collection: Filebeat
 2. Log Processing & Parsing: Logstash
 3. Log Storage & Search: Elasticsearch
 4. Visualization & Dashboards: Kibana

Core Database Design

- MongoDB with collections: Users, Buses, Trips, Bookings, Ratings
- MongoDB Sessions for transactional consistency
- Optimistic locking for seat booking
- Rollbacks for failed payments/bookings

Business Logic

- Real-time seat locking on booking attempt
- Automatic operator verification by admin
- Booking cancellation within configurable time window

API Design

Authentication:

- POST /users/register - User registration
- POST /users/login - User authentication
- POST /users/logout - User logout
- GET /users/profile - Get user profile
- PUT /users/profile - Update user profile

Trip Management:

- GET /trips?source&destination&date - Search trips

- POST /trips - Create trip (operators)
- GET /trips/:id - Get trip details
- PUT /trips/:id - Update trip (operators)
- DELETE /trips/:id - Delete trip (operators)

Booking Management:

- POST /bookings - Create booking
- GET /bookings/user - Get user bookings
- POST /bookings/:id/cancel - Cancel booking

Bus Management:

- POST /buses - Create bus (operators)
- GET /buses - Get operator buses
- PUT /buses/:id - Update bus (operators)
- DELETE /buses/:id - Delete bus (operators)

Admin Management:

- GET /admin/users - Get all users
- GET /admin/operators - Get all operators
- PUT /admin/block/:userId - Block user
- PUT /admin/unblock/:userId - Unblock user
- PUT /admin/verify-operator/:userId - Verify operator

b. Test Plan

- **Unit Testing:** Jest for each service layer
- **Integration Testing:** Booking workflows, seat locking
- **Load Testing:** Using K6 to simulate concurrent bookings
- **Security Testing:** JWT validation, role abuse prevention

c. Monitoring & Alerting

- **Log Collection:** Filebeat
- **Log Parsing:** Logstash
- **Search & Storage:** Elasticsearch
- **Visualization:** Kibana dashboards
- **Alerts:** Custom rules on booking failures, payment errors

d. Rollback Plan

- Tag-based Docker image rollback
- MongoDB document versioning for bookings
- Rollback scripts for trip/bus deletions